Homework 11: Implementation of PCF

CS3490: Programming Languages

The Language PCF

In this assignment, you will implement a variation of the language known as PCF. This is a strongly typed, Turing-complete, functional programming language. It is based on the simply typed lambda calculus, extending it with a native type of integers and a mechanism that allows arbitrary recursive definitions. The primitive operators include negation and a conditional that allows branching on whether the given integer expression is positive. From this, all other computable operations can be defined.

The precise definition of this language is as follows.

1. Syntax

1.1. Abstract Syntax

The *types* are generated from a single base type $\mathbb{Z}$, representing the type of integers (both positive and negative) using the function type constructor:

$$\mathbb{T} \quad ::= \quad \mathbb{Z} \mid \mathbb{T} \to \mathbb{T}$$

The *terms* are generated from variables and integer constants using application, abstraction, subtraction, and IfPos — an if-then-else construct based on testing whether a given number is positive. There is also a $\mathtt{Y}$-combinator that enables recursive definitions.

$$\Lambda \quad ::= \quad \mathbb{V} \mid \Lambda\Lambda \mid \lambda\mathbb{V}\Lambda \mid \underline{\mathbb{Z}} \mid \Lambda - \Lambda \mid \mathsf{IfPos}(\Lambda, \Lambda, \Lambda) \mid \mathtt{Y}$$

1.2. Haskell representation

The above grammars for types and terms are encoded in Haskell using datatype definitions — analogous to the encoding of arithmetic/boolean expressions and instructions.

You should try to write these definitions in Haskell yourself, and check your code against that given on the next page. On the final exam, the language will be extended with new type and term constructors, and the first question will be to translate them from the abstract syntax to the Haskell code. Use this moment to practice doing this!

```
type Vars = String
type TVars = String
--     T  ::=  Z | T -> T | TV
data Types = Ints | Fun Types Types | TVar TVars
  deriving (Show,Eq)
--    /\  ::= \/ | /\ /\ | \ \/ /\ | C Int | /\ - /\ | IfPos(/\,/\,/\) | Y
data Terms = Var Vars | App Terms Terms | Abs Vars Terms
           | Num Integer | Sub Terms Terms | IfPos Terms Terms Terms | Y
  deriving (Show,Eq)
```

Here `TVar` was added as an auxiliary type constructor to be used during type inference.

2. Parsing and Lexical Analysis

2.1. Question (10 pts)

Define the following lexical grammar

```
data Token = VSym String | CSym Integer
           | SubOp | IfPositiveK | ThenK | ElseK | YComb
           | LPar | RPar | Dot | Backslash
           | Err String | PT Terms
           deriving (Eq,Show)
```

Write a function `lexer :: String -> [Token]` which takes a textual representation of a term and generates a list of tokens. The lexical rules are as follows:

- A *variable* is any string which *starts with a lowercase letter* and is followed by *zero or more alphanumeric characters*.

  Examples of variables: `x y xy x1 xY x1z5 xXzZ`

- A *constant* is a string which consists of one or more digits.

  To make parsing easier, only positive numbers will be supported at the input level.

  Negative numbers will be entered by subtracting them from zero. For example, -3 can be entered as `0-3`.

- The subtraction operator is represented by a minus: `-`.

- IfPos-operator is represented on the input level by the keywords `ifPositive`, `then`, and `else`. Note that there is no separate type for keywords; they are "inlined" into the Token type.

- The Y-combinator is represented by `Y`.

- Note that the keywords begin with a lower-case letter, and the fixed point combinator is an upper-case letter.

- The punctuation tokens are self-explanatory.

- `Err` and `PT` are auxiliary tokens to be used internally by the parser for error flagging and keeping track of parsed subterms.

2.2. Question (10 pts)

Write a function `parser :: [Token] -> Either Terms String` that takes a list of tokens and attempts to produce a term out of them.

If no parse is possible, it should output `Right e` with `e :: String` being the error message identifying the error.

Otherwise, it should output `Left t`, where `t :: Terms`.

Unlike the parsers in the previous assignments, this function should assume that the input string has already been ran through the lexer.

(*Hint.* Use a shift–reduce helper function `sr` following the examples done in class.)

The rules for parsing should follow the standard conventions for writing lambda terms on paper, as described below.

- Variables and constants (including `Y`) appear on their own with no additional signs.
- Application is written by juxtaposition. Two terms M and N appearing next to each other denote the application term MN.
- Abstraction is written in the form `\x.M`: backslash, a variable, a dot, and a term.
- Subtraction is written in the standard infix notation.
- $\mathsf{IfPos}(s,t,u)$ is written as: `ifPositive` s `then` t `else` u, where s,t,u are terms.

The lexical errors should be reported as such, and the first 10 characters starting from the invalid symbol included in the error string.

Parse errors should be reported as such, and the final state of the parse stack included in the error string.

3. Typing

Recall that the syntax of PCF terms and types:

$$\begin{array}{lcl} \mathbb{T} & ::= & \mathbb{Z} \mid \mathbb{T} \to \mathbb{T} \\ \Lambda & ::= & \mathbb{V} \mid \Lambda\Lambda \mid \lambda\mathbb{V}\Lambda \mid \underline{\mathbb{Z}} \mid \Lambda - \Lambda \mid \mathsf{IfPos}(\Lambda,\Lambda,\Lambda) \mid \mathtt{Y} \end{array}$$

The typing rules specify the types allowed for each term constructor.

They are given in the figure below.

$$\frac{(x:A)\in\Gamma}{\Gamma\vdash x:A}\ \text{Var} \qquad \frac{\Gamma, x:A\vdash r:B}{\Gamma\vdash \lambda x.r : A\to B}\ \text{Abs} \qquad \frac{n\in\{0,1,-1,2,-2,\dots\}}{\Gamma\vdash \underline{n}:\mathbb{Z}}\ \text{Num}$$

$$\frac{\Gamma\vdash s:A\to B \qquad \Gamma\vdash t:A}{\Gamma\vdash st:B}\ \text{App} \qquad \frac{\Gamma\vdash s:\mathbb{Z} \qquad \Gamma\vdash t:\mathbb{Z}}{\Gamma\vdash s-t:\mathbb{Z}}\ \text{Sub}$$

$$\frac{\Gamma\vdash r:\mathbb{Z} \qquad \Gamma\vdash s:A \qquad \Gamma\vdash t:A}{\Gamma\vdash \mathsf{IfPos}(r,s,t):A}\ \text{IfPos} \qquad \frac{}{\Gamma\vdash \mathtt{Y}:(A\to A)\to A}\ \text{Y-Comb}$$

Define the following type synonyms:

```
type Constr =  (Types,Types)
type Cxt = [(Vars,Types)]
type TSub   = [(TVars,Types)]
```

3.1. Question (10 pts)

Implement the functions that perform substitutions on types and constraints, and obtain all the free variables occurring in types and contexts.

```
tsubst :: (TVars,Types) -> Types  -> Types
csubst :: (TVars,Types) -> Constr -> Constr
getTVars    :: Types -> [TVars]
getTVarsCxt :: Cxt   -> [TVars]
```

3.2. Question (20 pts)

Implement a function `genConstrs :: Cxt -> Terms -> Types -> [Constr]` which generates a list of constraints for a given context, term, and target type.

You should use the typing rules given in the table above to direct the implementation code of this function. Remember that the Hindley–Milner type inference algorithm is *syntax-directed*: which rule you should use is completely determined by which term you are trying to type.

(On the final exam, there will be a similar question that will likewise be worth more points than other questions.)

3.3. Question (10 pts)

Implement the following functions

```
unify :: [Constr] -> [(TVars,Types)]
infer :: Terms -> Types
```

The function `unify` takes a list of constrains and produces a type substitution, assigning type variables to types. It can raise an error in case of a circular constraint (the occurs check) or a type error (e.g., a function type being equated to an integer).

The function `infer` takes a fully parsed PCF term, and computes the most general type for that term. *Unlike the code in the class, it is assumed that the term has already been ran through the lexer and the parser.*

4. Reduction

In this section, you will implement the functions that reduce a term to its normal form.

4.1. Question (10 pts)

First, you will need to implement a function `subst :: (Vars,Terms) -> Terms -> Terms`, where
`subst (x,t) s` replaces every occurrence of the variable `x` in the term `s` by the term `t`.

4.2. Question (20 pts)

Next, you are to implement the function `red :: Terms -> Terms` that performs one step of parallel reduction.

This function should recursively descend down the term until it finds a pattern that matches the left side of one of the *rewrite rules*.

For this version of PCF, these rules are as follows.

4.2.1. Reduction rules

$$\begin{aligned}
(\lambda x.s)t &\longrightarrow s[t/x] \\
\underline{m} - \underline{n} &\longrightarrow \underline{m-n} \\
\mathsf{IfPos}(\underline{n}, s, t) &\longrightarrow \begin{cases} s & n > 0 \\ t & n \leq 0 \end{cases} \\
\mathtt{Y}t &\longrightarrow t(\mathtt{Y}t)
\end{aligned}$$

4.2.2. Congruence rules

If the term does *not* match the left side of any of these rules, the function should proceed recursively down into the subterms, without changing the topmost node on the parse tree of the term.

These cases of the function are known as *congruence laws*. They can be summarized using the following inference rules, presented on the level of abstract syntax.

$$\frac{x \in \mathbb{V}}{x \Rightarrow x} \qquad \frac{n \in \mathbb{Z}}{n \Rightarrow n} \qquad \frac{r_0 \Rightarrow r_1 \qquad s_0 \Rightarrow s_1 \qquad t_0 \Rightarrow t_1}{\mathsf{IfPos}(r_0, s_0, t_0) \Rightarrow \mathsf{IfPos}(r_1, s_1, t_1)}$$

$$\frac{r_0 \Rightarrow r_1}{\lambda x.r_0 \Rightarrow \lambda x.r_1} \qquad \frac{s_0 \Rightarrow s_1 \qquad t_0 \Rightarrow t_1}{s_0 t_0 \Rightarrow s_1 t_1} \qquad \frac{s_0 \Rightarrow s_1 \qquad t_0 \Rightarrow t_1}{s_0 - t_0 \Rightarrow s_1 - t_1} \qquad \frac{}{\mathtt{Y} \Rightarrow \mathtt{Y}}$$

4.2.3. Base cases

The above table also summarizes the rules at which the function should return without any reduction steps being performed. This happens precisely when the input term is a variable or a constant terms. (In this case, there is no computation to be done.)

To summarize, if `m :: Terms` is an internal representation of the term M, and $M \Rightarrow N$, then `red m` should produce an internal representation of the term N.

The basic behavior of the `red` function should be:

- If a given term can be reduced at the top level, do it and return the result.
- If a given term cannot be reduced anymore (constant or variable), return it.
- If a term is neither reducible at the top level, nor is a leaf of the syntax tree, then recursively reduce its immediate subterms in parallel.

Note. This question will also be present on exam, and it will be worth more points than average due to the sheer length of writing the many cases of the `red` function.

4.2.4. Multistep reduction

Finally, implement the function `reds :: Terms -> Terms` that calls `red` recursively on the input until no more progress is made. Specifically, once the output produced by `red` is the same as the input, this means that the normal form has been reached or we have entered a cycle. The resulting term can then be returned. (Otherwise, `reds` should be called again on the output of `red`.)

In abstract terms, this function implements the reflexive–transitive closure of the $\Rightarrow$ relation, denoted $\Rightarrow^*$. It can be defined using the following inference rules.

$$\frac{}{s \Rightarrow^* s} \qquad \frac{r \Rightarrow s \qquad s \Rightarrow^* t}{r \Rightarrow^* t}$$

Examples

Let $\mathtt{I} = \lambda x.x$ and $\mathtt{K} = \lambda x.\lambda y.x$. Then

- $\mathtt{I}\underline{5} \to_0 \underline{5}$, $\mathtt{YI} \to_0 \mathtt{I}(\mathtt{YI})$, $\underline{5} - \underline{2} \to_0 \underline{3}$.

  Each of these steps can be performed by applying one of the reduction rules given on page 1 at the very top of the term.

- $\mathtt{K}(\mathtt{I}\underline{5}) \not\to_0 \mathtt{K}\underline{5}$, $\lambda z.z - (\underline{5} - \underline{2}) \not\to_0 \lambda z.z - \underline{3}$.

  Neither of these steps can be performed at the very top of the term. However,

- $\mathtt{K}(\mathtt{I}\underline{5}) \Rightarrow \mathtt{K}\underline{5}$, $\lambda z.z - (\underline{5} - \underline{2}) \Rightarrow \lambda z.z - \underline{3}$.

  Both of them can be performed once congruence rules are added.

- $\mathsf{IfPos}(\mathtt{I}\underline{5}, \mathtt{YI}, \underline{5} - \underline{2}) \Rightarrow \mathsf{IfPos}(\underline{5}, \mathtt{I}(\mathtt{YI}), \underline{3})$.

  Also, $\mathsf{IfPos}(\underline{5}, \mathtt{I}(\mathtt{YI}), \underline{3}) \Rightarrow \mathtt{I}(\mathtt{YI})$. And $\mathtt{I}(\mathtt{YI}) \Rightarrow \mathtt{YI}$.

  Parallel subterms can be contracted simultaneously by $\Rightarrow$ if they do not overlap.

- $\mathsf{IfPos}(\mathtt{I}\underline{5}, \mathtt{YI}, \underline{5} - \underline{2}) \not\Rightarrow \mathtt{YI}$.

  $\Rightarrow$ is limited to a single parallel step only.

- $\mathsf{IfPos}(\mathtt{I}\underline{5}, \mathtt{YI}, \underline{5} - \underline{2}) \Rightarrow^* \mathtt{YI}$.

  $\Rightarrow^*$ allows chaining together any number (including 0) of $\Rightarrow$-steps.

5. Frontend

5.1. Question (10 pts)

Implement the `main` method which will perform the following tasks when ran:

- query the user for the input file name,
- load the file into a string
- attempt to parse the string into a term.
- if failed, display an error message and quit
- attempt to infer a type for the parsed term
- if failed, the program can throw an error message
- otherwise, display the computed type
- attempt to reduce the term to normal form
- if successful, display the final result and quit